Buff 系统交接文档

> **交接对象**：角色脚本 & 敌人脚本负责人 **文档目的**：快速了解 Buff 系统的 API、事件和集成方式，无需阅读全部源码即可接入使用

---

1. 系统概览

Buff 系统采用 **ScriptableObject 配置 + 单例管理器 + 事件中心** 架构，不通过 Unity 组件附加到角色/敌人身上。Buff 的效果通过 `BuffManager.Instance.GetTotalStatModifier()` 查询得出数值，角色/敌人的属性系统在计算最终属性时应调用该方法。

核心文件速查

用途	文件路径
管理器（API 入口）	`Assets/Scripts/Data/Runtime/BuffManager.cs`
运行时实例	`Assets/Scripts/Data/Runtime/BuffInstance.cs`
Buff 配置 SO	`Assets/Scripts/Data/Config/BuffConfigSO.cs`
属性修改器	`Assets/Scripts/Data/Config/StatModifier.cs`
Buff 掉落池 SO	`Assets/Scripts/Data/Config/BuffPoolSO.cs`
场景掉落物	`Assets/Scripts/Rogue/Dungeon/Reward/BuffDrop.cs`
事件定义	`Assets/Scripts/Core/Events/GameEvents.cs`
事件类型枚举	`Assets/Scripts/Core/Events/EventType.cs`
枚举定义	`Assets/Scripts/Core/Buff/` 目录下（StatType / DurationType / ModifyType / Rarity）
运行时验证器	**`Assets/Scripts/Debug/BuffSystemVerifier.cs`**

---

2. 运行时脚本

2.4 脚本中内置的角色属性计算公式

验证脚本中展示了角色脚本接入 Buff 系统的标准模式：

```
// 最终属性 = (基础值 + 所有 Fixed 修正之和) × (1 + 所有 Percent 修正之和)
float flatBonus  = BuffManager.Instance.GetTotalStatModifier(statType,
ModifyType.Flat);
float pctBonus   = BuffManager.Instance.GetTotalStatModifier(statType,
ModifyType.Percent);
float finalValue = (baseValue + flatBonus) * (1f + pctBonus);
```

---

3. BuffManager 核心 API

> **单例访问：** `BuffManager.Instance`

3.1 属性查询（角色/敌人必须调用）

```
// 查询所有激活 Buff 对指定属性的总修正值
float total = BuffManager.Instance.GetTotalStatModifier(StatType stat, ModifyType
type);
```

示例： 计算角色最终攻击力

```
float baseAttack = 10f;
float flatBonus = BuffManager.Instance.GetTotalStatModifier(StatType.Attack,
ModifyType.Flat);
float percentBonus = BuffManager.Instance.GetTotalStatModifier(StatType.Attack,
ModifyType.Percent);
float finalAttack = (baseAttack + flatBonus) * (1f + percentBonus);
```

示例： 计算角色最终移动速度

```
float baseSpeed = 5f;
float flatBonus = BuffManager.Instance.GetTotalStatModifier(StatType.MoveSpeed,
ModifyType.Flat);
float percentBonus = BuffManager.Instance.GetTotalStatModifier(StatType.MoveSpeed,
ModifyType.Percent);
float finalSpeed = (baseSpeed + flatBonus) * (1f + percentBonus);
```

3.2 状态查询

```
bool hasBuff = BuffManager.Instance.HasBuff("buff_attack_flat");         // 是否存
在
BuffInstance buff = BuffManager.Instance.GetBuff("buff_attack_flat");    // 获取实
例（含层数等）
BuffInstance buff2 = BuffManager.Instance.FindBuff("buff_attack_flat");  // 同上
```

```
（兼容旧名）
IReadOnlyList<BuffInstance> all = BuffManager.Instance.GetActiveBuffs(); // 全部激活 Buff
int count = BuffManager.Instance.ActiveBuffCount;                        // 激活数量
```

3.3 Buff 应用与移除

```
// 通过 buffId 应用
BuffManager.Instance.ApplyBuff("buff_attack_flat", sourceId: "SkillCast");

// 通过 BuffConfigSO 直接应用
BuffManager.Instance.ApplyBuff(myBuffConfig, sourceId: "ItemUse");

// 移除指定 Buff
BuffManager.Instance.RemoveBuff("buff_attack_flat");
```

> **sourceId 说明**：标识 Buff 来源的字符串，目前使用 `"RewardDrop"` 表示奖励掉落，自定义时可用 `"SkillCast"`, `"ItemUse"` 等。用于日志和事件追踪，不影响 Buff 行为。

3.4 BuffInstance 字段说明

```
[Serializable]
public class BuffInstance
{
    public string BuffId;        // 对应 BuffConfigSO.buffId
    public int StackCount;       // 当前叠加层数（≥1）
    public float RemainingTime;  // 剩余时间（秒），仅 Timed 类型有效
    public int RemainingRooms;   // 剩余房间数，仅 RoomScoped 类型有效
    public string SourceId;      // 来源标识
}
```

---

4. Buff 持续类型（DurationType）

角色/敌人脚本在监听 Buff 事件时，需理解五种持续时间的行为差异：

类型	枚举值	生命周期	叠加行为
`Permanent`	0	整局有效	指数递减叠加（见 §8）
`Timed`	1	N 秒后过期	重获时刷新剩余时间
`RoomScoped`	2	N 个房间后过期	重获时重置房间计数
`Stack`	3	按层数存在	重获增加层数
`Instant`	4	立即生效后丢弃	不进入 ActiveBuffs 列表

---

5. 可修改的属性类型（StatType）

枚举值	含义
`MaxHP` (0)	生命上限
`Attack` (1)	攻击力
`Defense` (2)	防御力
`MoveSpeed` (3)	移动速度
`AttackSpeed` (4)	攻击速度
`CritRate` (5)	暴击率
`CritDamage` (6)	暴击伤害

5.1 修改方式（ModifyType）

枚举值	含义
`Flat` (0)	固定值加减（如 +5 攻击力）
`Percent` (1)	百分比修正（如 +20% 移动速度）

---

6. 事件系统

> 事件通过 `EventCenter` 广播，使用 `RogueDungeon.Core.Events` 命名空间

6.1 Buff 相关事件

事件类型	Payload	触发时机
`BuffApplied`	`{ BuffId, SourceId, StackCount }`	Buff 应用成功后
`BuffExpired`	`{ BuffId, Duration }`	Buff 过期/被移除
`BuffStackChanged`	`{ BuffId, OldStack, NewStack }`	叠加层数变化时
`RewardClaimed`	`{ BuffId, RoomId }`	奖励被领取

6.2 事件监听示例

```
using RogueDungeon.Core.Events;

// 监听 Buff 应用
EventCenter.AddListener<BuffAppliedEvent>(GameEventType.BuffApplied,
OnBuffApplied);

// 监听 Buff 过期
EventCenter.AddListener<BuffExpiredEvent>(GameEventType.BuffExpired,
OnBuffExpired);
```

```
private void OnBuffApplied(BuffAppliedEvent evt)
{
    // 根据 evt.BuffId 更新角色属性
    // 例如：刷新 UI、播放特效、重新计算属性
}

private void OnBuffExpired(BuffExpiredEvent evt)
{
    // Buff 移除后重新计算属性
}

// 别忘了在 OnDestroy 中移除监听
private void OnDestroy()
{
    EventCenter.RemoveListener<BuffAppliedEvent>(GameEventType.BuffApplied,
OnBuffApplied);
    EventCenter.RemoveListener<BuffExpiredEvent>(GameEventType.BuffExpired,
OnBuffExpired);
}
```

---

7. Buff 掉落与拾取流程

```
房间清空 → RewardSpawner 生成 BuffDrop 预制体 → 玩家触碰 Collider2D
→ BuffDrop.OnTriggerEnter2D() → BuffManager.ApplyBuff(buffId, "RewardDrop")
→ 广播 BuffApplied 事件 → 角色脚本监听到事件 → 重新计算属性
```

- 掉落物通过对象池 (`ObjectPool`, key=`"BuffDrop"`) 管理，无需手动创建/销毁
- `ExclusivePickupGroup` 确保多个候选奖励只能选一个
- 每次拾取后自动保存 RunState（含 ActiveBuffs 列表）

---

8. 永久型 Buff 叠加公式（Permanent 指数衰减）

永久型 Buff 重复获取时按指数衰减叠加，公式为：

```
有效倍率 = 1 + decayRate + decayRate² + ... + decayRate^(stackCount-1)
有效值 = modifier.Value × 有效倍率
```

示例（decayRate = 0.7, 基础值 = 5）：

- 1 层：`5 × 1.0 = 5`
- 2 层：`5 × (1 + 0.7) = 8.5`
- 3 层：`5 × (1 + 0.7 + 0.49) = 10.95`

此计算由 `BuffManager.GetEffectiveValue()` 内部处理，外部只需调用 `GetTotalStatModifier()` 即可拿到最终值。

---

9. 接入清单（角色/敌人脚本需要做的事）

必须接入

1. **属性计算时调用 BuffManager**
   - 在获取角色/敌人的最终攻击力、速度、防御等属性时，调用 `GetTotalStatModifier()` 叠加 Buff 修正值
   - 建议在属性计算函数中集中调用，避免散落各处
2. **监听 Buff 事件以刷新属性**
   - 监听 `BuffApplied`：Buff 应用后重新计算属性
   - 监听 `BuffExpired`：Buff 过期后重新计算属性
   - 监听 `BuffStackChanged`：层数变化后重新计算属性（可选，通常 BuffApplied 已覆盖）
3. **确保调用时机正确**
   - `BuffManager` 是 DontDestroyOnLoad 单例，在 `RunInit` 阶段初始化
   - 不要在 `Awake` 中过早调用 Buff 查询（可能在 Run 初始化之前）
   - 建议在 `Start()` 或 `OnEnable()` 中首次查询，或在监听到 `RunInit` / `BuffApplied` 事件后查询

可选接入

4. **UI 显示**
   - 监听 Buff 事件更新 HUD 上的 Buff 图标列表
   - 可使用 `GetActiveBuffs()` 获取当前所有 Buff 实例
   - 通过 `BuffManager.Instance.BuffPool.FindByBuffId(buffId)` 获取 Buff 的 icon、displayName、description
5. **Buff 来源扩展（技能/道具）**
   - 如需让技能或道具也能施加 Buff，直接调用 `BuffManager.Instance.ApplyBuff(buffId, sourceId)`
   - 只需传入 buffId 字符串，无需引用 BuffConfigSO 资产

---

10. Buff 配置创建流程

> 策划/设计人员通过 Unity Editor 创建新 Buff，无需写代码

1. 在 Project 窗口右键 → `Create` → `Data` → `Buff Config`
2. 填写字段：
   - `Buff Id`：唯一标识符（如 `buff_fire_damage`）
   - `Display Name` / `Description`：显示用
   - `Rarity`：稀有度
   - `Duration`：持续类型

    - `Duration Value`：持续值（秒/房间数）
    - `Decay Rate`：永久型叠加衰减率（0~1）
    - `Max Stack`：最大叠加层数（0=无限）
    - `Modifiers`：属性修改列表，每条包含 Stat / Type / Value
3. 将新 Buff 添加到 `BuffPool_Default` 池中（`Assets/Data/Buff/BuffPool_Default.asset`）

---

11. 关键约定与注意事项

- **命名空间**：Buff 系统使用 `RogueDungeon.Core.Buff`、`RogueDungeon.Data.Runtime`、`RogueDungeon.Data.Config`、`RogueDungeon.Core.Events`
- **无 Component 模式**：Buff 不通过 Unity MonoBehaviour 附加到角色，所有效果通过 `BuffManager` 查询
- **buffId 是全局唯一键**：所有 Buff 操作以 buffId 字符串为核心标识
- **即时型 Buff（Instant）**：不进入 ActiveBuffs 列表，只广播一次 `BuffApplied` 事件。角色脚本需监听该事件来处理即时效果
- **BuffManager 初始化**：在 `RunInit` 游戏状态时初始化，依赖 `RunManager.Instance.CurrentRun`
- **序列化**：`BuffInstance` 是 `[Serializable]` 类，随 `RunState` 一起通过 JsonUtility 持久化。存档/读档自动包含 Buff 数据
- **对象引用**：`BuffManager._activeBuffs` 直接引用 `RunState.ActiveBuffs` 列表，修改会同步反映到存档数据

---

12. 常见问题

Q：为什么 `GetTotalStatModifier()` 返回 0？ A：检查是否在 Run 进行中（`RunManager.Instance.CurrentRun != null`），以及 BuffPool 是否已绑定。BuffManager 在 Hub 和 RunEnd 状态下不持有任何 Buff。

Q：如何让 Buff 在 N 秒后自动消失？ A：将 `Duration` 设为 `Timed`，`DurationValue` 设为秒数。BuffManager 的 `Update()` 会自动逐帧倒计时并移除。

Q：如何限制 Buff 最大叠加层数？ A：在 `BuffConfigSO` 中设置 `MaxStack`（0 表示无限）。

Q：能不能用代码直接移除一个 Buff？ A：调用 `BuffManager.Instance.RemoveBuff("buffId")` 即可。